QR Code Based Attendance System

Complete Build Guide, Architecture & Viva Preparation — Built on AWS at ₹0 cost

Made by Saurav

August 2026

QR Code Based Attendance System — Complete Build Guide

Made by Saurav

A working attendance system built on Amazon Web Services, designed to cost ₹0.

How to read this guide

Every step tells you five things:

1. **What** we are doing
2. **Why** we are doing it
3. **Exactly** what to click or type
4. **How to test** that it worked
5. **What usually goes wrong** and how to fix it

Do the steps in order. Do not skip Step 1 — that is the step that stops you getting a bill.

Time needed: about 2 hours the first time.

What you are building

A student opens a website on their phone, points the camera at a QR code on the teacher's screen, and their attendance is recorded. The QR code changes every 10 seconds, so a screenshot sent to an absent friend is useless by the time it arrives.

Part	What runs it
The website	Amazon S3 + CloudFront
Login	Amazon Cognito
The brain	AWS Lambda
The data	Amazon DynamoDB

Part	What runs it
Permissions	IAM
Error messages	CloudWatch Logs
Your money guard	AWS Budgets

The complete request flow

This is the part to memorise for your viva.

When the teacher starts a session:

```
Teacher clicks "Start session"
  → browser sends the request to the Lambda Function URL,
    with the login token Cognito gave it
  → Lambda checks the token and confirms the person is a Teacher
  → Lambda writes a new row into the Sessions table in DynamoDB
  → Lambda sends back the session ID
  → the teacher's browser now asks Lambda every 10 seconds for a fresh
code
  → Lambda builds a short signed code from
       (session ID + which 10-second window we are in)
    signed with a secret key only Lambda knows
  → the browser turns that text into a QR picture
  → the screen shows the QR with a countdown, then it changes
```

When a student scans:

```
Student scans the QR
  → browser reads the short code out of the picture
  → browser sends { code } to the Lambda Function URL,
    with the student's login token attached
  → Lambda runs six checks, all on the server:
       1. is the login token really signed by our Cognito?
(authentication)
       2. was this code really made by us?                    (anti-
forgery)
       3. is the code's 10-second window still current?
(expiry)
       4. is the session still open and not past its end time?
(session validity)
       5. is this student actually enrolled in this class?
(authorization)
       6. does an attendance row already exist for them?      (no
duplicates)
  → Lambda writes one row into the Attendance table
  → Lambda returns "success"
  → the phone shows "Attendance marked successfully"
  → the teacher's screen shows the name appear within seconds
```

Nothing the phone sends is trusted. Every check happens inside Lambda.

STEP 1 — Create your AWS account and your money guard

What: Open an AWS account, then immediately set an alarm that emails you if anything ever costs money.

Why: AWS bills you automatically. The alarm is free, takes four minutes, and is the difference between a ₹0 project and a nasty surprise.

1a. Read this before you sign up

AWS changed its free tier on 15 July 2025. Here is what a new account gets today:

- You choose a **Free plan** or a **Paid plan** at signup.
- You get **$100 in credits** immediately, plus up to **$100 more** for finishing onboarding tasks.
- The Free plan ends when **six months pass** or **the credits run out**, whichever comes first.
- On the Free plan, when that happens **the account closes automatically**. Running resources shut down. You then have 90 days to upgrade and get your data back.

Separately there are **"always free" services** with permanent monthly limits that never expire. Our project lives almost entirely inside those.

> **Plan around the six months.** If your demo is more than five months away, note the date you sign up and put a reminder in your phone.

A card is required at signup even on the Free plan. AWS may charge and refund about ₹2 to check the card is real.

1b. Sign up

1. Go to **aws.amazon.com → Create an AWS Account**
2. Email, account name (e.g. `saurav-college-project`), verify the code
3. Set a strong root password
4. Account type: **Personal**
5. Enter your address and phone number
6. Add your card
7. Verify your phone by SMS or call
8. Support plan: choose **Basic support — Free**. Never choose Developer or Business, those cost money.

1c. Lock the root account (2 minutes, do not skip)

The root login can do anything, including spending money. Protect it.

1. Sign in → click your name (top right) → **Security credentials**
2. Under **Multi-factor authentication (MFA)** → **Assign MFA device**
3. Choose **Authenticator app**, scan the QR with Google Authenticator or Authy
4. Enter two codes to finish

1d. Turn on billing alerts

1. Click your name (top right) → **Billing and Cost Management**
2. Left menu → **Billing preferences**
3. Tick **Receive AWS Free Tier alerts** and enter your email
4. Tick **Receive billing alerts** (or "Receive CloudWatch billing alerts")
5. **Save preferences**

1e. Create a $1 budget (the important one)

1. Still in Billing → left menu → **Budgets** → **Create budget**
2. Choose **Customize (advanced)** → **Cost budget**
3. Name: `zero-cost-guard`
4. Period: **Monthly**, Budget renewal type: **Recurring**
5. Budgeting method: **Fixed**, Budgeted amount: **1.00**
6. Next → **Add an alert threshold**
   - Threshold: **1** percent of budgeted amount (so it fires at ₹1)
   - Email recipient: your email
7. Next → **Create budget**

The first two budgets are free.

Test it worked

- Billing → Budgets shows `zero-cost-guard` with **$0.00** spent
- You get an AWS email confirming the budget was created
- Signing out and back in asks for your authenticator code

Common problems

Problem	Fix
"Your account is being verified"	Normal. Can take up to 24 hours. Wait before continuing.
No Billing menu	You are signed in as an IAM user, not root. Sign in as root.
Card declined	Indian cards sometimes block international charges. Use a different card or enable international transactions in your banking app.

STEP 2 — Choose your region and never change it

What: Pick one AWS region and do everything in it.

Why: Free allowances are counted **per region** for some services. If you accidentally build half the project in Mumbai and half in Virginia, you can use two lots of capacity and get charged. It also breaks things in confusing ways.

Do this: Click the region name at the top right of the console and choose **Asia Pacific (Mumbai) ap-south-1**.

Every single time you open the console from now on, glance at that box first. If it does not say Mumbai, change it before doing anything.

> CloudFront is the one exception — it is global and has no region. That is normal.

STEP 3 — Create the database

What: Five DynamoDB tables.

Why: We need somewhere to keep users, classes, who is in which class, attendance sessions, and attendance records.

The design

Table	Key	What it holds
`att_Users`	userId	Name, email, role. Copied from Cognito on first login so a teacher can search by email.
`att_Classes`	classId	Class name, subject, teacher, 6-letter join code
`att_Enrollments`	classId + studentId	Which student belongs to which class
`att_Sessions`	sessionId	One lecture's attendance window: start, end, open or closed
`att_Attendance`	sessionId + studentId	One row = one student present in one session

The most important line in the whole project is that last one. Because the key is the *pair* `sessionId + studentId`, that pair can physically only exist once. Duplicate attendance is not blocked by clever code — it is blocked by the shape of the database.

The cost rule you must not get wrong

DynamoDB has two billing modes. The free allowance of 25 read units and 25 write units applies **only to provisioned capacity mode**. Tables set to **on-demand mode are billed per request from the very first read or write**, with no free offset.

On-demand is the friendlier-looking option in the console. **Do not use it.**

Three more rules that go with it:

- The 25 units are shared across **every table in your account in that region**, not 25 each.
- **Every secondary index needs its own capacity too.** Table units + all index units must stay under 25.
- The table class must be **Standard**. Standard-IA is not free-tier eligible and charges from the moment you create it.

Our script uses **20 read and 20 write** in total, leaving 5 spare.

Do this

1. Check the region box says **Mumbai**
2. Click the **terminal icon** in the top bar to open **AWS CloudShell** (free, comes with the AWS command line already installed)
3. Wait for it to say "Preparing your terminal"
4. Open `aws/01-create-tables.sh` from this project, copy the **whole file**, paste into CloudShell, press Enter

It takes about a minute and prints each table as it becomes ready.

Test it worked

- Console → search **DynamoDB** → **Tables**
- You see five tables, all **Active**
- Click `att_Attendance` → the **Indexes** tab shows `student-index`
- Click any table → **Additional settings** → capacity mode says **Provisioned**

Common problems

Problem	Fix
`ResourceInUseException`	The table already exists. Safe to ignore, or delete it and run again.
Tables in the wrong region	CloudShell uses whatever region the console is in. Fix the region, then delete and recreate.
`AccessDenied`	You are signed in as an IAM user without permissions. Sign in as root for setup.

STEP 4 — Create the login system

What: A Cognito user pool, an app client, and two groups.

Why: Cognito handles passwords, email confirmation, password reset and secure tokens. Writing that yourself is where student projects get hacked in front of the examiner.

Cost: Cognito's free tier covers 10,000 monthly active users and it does not expire at the end of the 12-month term. You will have about five users. We turn **MFA off and never enable SMS**, because SMS messages cost real money per message.

Do this

Open **CloudShell** again, copy the whole of `aws/02-create-cognito.sh`, paste, press Enter.

At the end it prints three lines. **Copy them into a notepad right now** — you need them twice.

```
REGION:         "ap-south-1",
USER_POOL_ID:   "ap-south-1_XXXXXXXXX",
APP_CLIENT_ID: "xxxxxxxxxxxxxxxxxxxxxxxxxx",
```

Understanding what that script did

- **User pool** = the list of users. Sign-in is by email. Passwords must be 8+ characters with upper case, lower case, a number and a symbol.
- **App client** = the doorway your website uses. We created it with **no client secret**, because a secret cannot be kept safe inside a web page. We enabled `ALLOW_USER_PASSWORD_AUTH` so the site can send an email and password directly.
- **Groups** = `Teacher` and `Admin`. Anyone not in a group is treated as a student. The group name travels inside the login token, and Lambda reads it to decide what the person is allowed to do.

Test it worked

Console → search **Cognito** → **User pools** → you see `qr-attendance-pool`. Open it → **App integration** tab → one app client called `qr-attendance-web`. Open it and confirm **Client secret: not generated** and that **ALLOW_USER_PASSWORD_AUTH** is ticked.

Common problems

Problem	Fix
`InvalidParameterException` about the schema	Someone already made a pool with the same name. Change the name in the script and run again.
Verification emails never arrive	Check spam. Cognito's built-in email sender is limited to about 50 messages a day — plenty for a demo, but do not test with 60 accounts.

STEP 5 — Create the backend brain

What: One Lambda function containing the whole backend.

Why: Lambda runs your code only when someone calls it and charges nothing when idle. Its free allowance of 1 million requests and 400,000 GB-seconds per month is in the **always-free** category, not a 12-month trial. Your whole project will use roughly 3% of one month's allowance.

We use **one** function rather than twelve, because one function means one place to paste code, one place to check for errors, and one log group. It costs exactly the same.

5a. Create the function

1. Console → search **Lambda** → **Create function**
2. Choose **Author from scratch**
3. Function name: `qr-attendance-api`
4. Runtime: **Node.js 22.x**
5. Architecture: **arm64** (slightly cheaper, same free allowance)
6. Leave permissions on **Create a new role with basic Lambda permissions**
7. **Create function**

5b. Paste the code

1. Scroll to the **Code source** editor
2. In the file tree on the left, the file is called `index.mjs`
3. Click it, select everything inside (Ctrl+A), delete it
4. Open `backend/index.mjs` from this project, copy the **whole file**, paste it in
5. Click **Deploy** (or Ctrl+S)

Wait for the green "Successfully updated" message.

5c. Give it the settings it needs

Go to the **Configuration** tab → **Environment variables** → **Edit** → add these, one at a time:

Key	Value
`USER_POOL_ID`	the pool ID from Step 4
`APP_CLIENT_ID`	the client ID from Step 4
`QR_SECRET`	a long random string you invent, e.g. `saurav-qr-2026-8fJ3kLpQ9x`
`TBL_USERS`	`att_Users`
`TBL_CLASSES`	`att_Classes`
`TBL_ENROLLMENTS`	`att_Enrollments`
`TBL_SESSIONS`	`att_Sessions`
`TBL_ATTENDANCE`	`att_Attendance`
`ALLOWED_ORIGIN`	* for now — we tighten this in Step 9

Click **Save**.

`QR_SECRET` is the key that signs every QR code. Anyone who knows it could make fake QR codes, so never put it in the frontend and never post it online.

Then **Configuration → General configuration → Edit**:

- **Timeout:** change 3 seconds to **10 seconds**
- **Memory:** leave at 128 MB

Save.

5d. Let Lambda talk to DynamoDB

Right now the function has no permission to read your tables. This is IAM doing its job.

1. **Configuration → Permissions →** click the **Role name** link (opens IAM)
2. **Add permissions → Create inline policy**
3. Click the **JSON** tab
4. Open `aws/lambda-dynamodb-policy.json` from this project and paste its contents
5. Replace `REGION` with `ap-south-1` and `ACCOUNT_ID` with your 12-digit account number (top right of the console, under your name). Delete the `"Comment"` line.
6. **Next →** policy name `qr-attendance-dynamodb` **→ Create policy**

This policy lets the function touch **only these five tables** and nothing else in your account. That is the "secure database access" requirement, and it is worth saying out loud in your viva.

Test it worked

In the Lambda console, click **Test →** create a new test event:

- Event name: `health`
- Paste this as the event JSON:

```
{ "rawPath": "/api/health", "requestContext": { "http": { "method":
"GET" } } }
```

Click **Test**. You should get status 200 and a body containing `"madeBy":"Saurav"`.

Common problems

Problem	Fix
`Cannot find module '@aws-sdk/client-dynamodb'`	Your runtime is Node.js 16 or older. Change it to Node.js 22.x.
`Runtime.UserCodeSyntaxError`	The paste was incomplete. Select all, delete, paste the whole file again, Deploy.
Test returns 500	Open **Monitor → View CloudWatch logs** and read the newest log stream. The real error is there.
`AccessDeniedException` on a table	The inline policy is missing, or REGION /

Problem	Fix
	ACCOUNT_ID were not replaced.

STEP 6 — Give the backend a web address

What: Turn on a Lambda Function URL.

Why: Your website needs an https address to send requests to. A Function URL is a permanent https endpoint attached directly to the function.

Why not API Gateway? API Gateway's free allowance of one million calls is a **12-month offer** and current pricing guidance says it applies only to accounts created before 15 July 2025; newer accounts draw down credits and then pay **$1.00 per million** HTTP API requests. Lambda Function URLs have **no additional charge** — you pay only the normal Lambda cost, which our project stays inside. Same result, one less service, one less thing that can bill you.

> If your college specifically requires API Gateway, you can add an **HTTP API** in front of this same Lambda later. At your traffic it would cost roughly ₹2 in total. The code does not need to change.

Do this

1. In your Lambda function → **Configuration** → **Function URL** → **Create function URL**
2. Auth type: **NONE** *(This does not mean "no security". It means AWS does not check the caller — our own code checks the Cognito login token on every request. That check is in section 3 of the code.)*
3. Tick **Configure cross-origin resource sharing (CORS)**
4. Allow origin: *
5. Allow headers: `content-type, authorization`
6. Allow methods: `GET, POST, OPTIONS`
7. **Save**

Copy the URL it gives you. It looks like:

```
https://abcd1234efgh5678.lambda-url.ap-south-1.on.aws/
```

Remove the trailing slash before you use it.

Test it worked

Paste this into your browser address bar (your URL + `/api/health`):

```
https://abcd1234efgh5678.lambda-url.ap-south-1.on.aws/api/health
```

You should see:

```
{"ok":true,"service":"QR Attendance API","madeBy":"Saurav"}
```

If you see that, your backend is live on the internet.

STEP 7 — Connect the website to AWS

What: Fill in four values in one file.

Why: Until now the website has been running in demo mode with fake data inside your browser. These four values switch it to the real thing.

First, see it working with no AWS at all

Before connecting anything, open `frontend/index.html` by double-clicking it. An orange **DEMO MODE** bar appears. Sign in with:

- Teacher: `teacher@demo.com` / `Demo@1234`
- Student: `student@demo.com` / `Demo@1234`

Play with it. Create a class, start a session, watch the QR rotate. This is exactly how the real thing behaves, and it means you always have a working demo even if the internet dies on presentation day.

Now connect it

Open `frontend/assets/config.js` in Notepad or VS Code and fill in all four values:

```
window.CONFIG = {
  REGION:         "ap-south-1",
  USER_POOL_ID:   "ap-south-1_XXXXXXXXX",
  APP_CLIENT_ID: "xxxxxxxxxxxxxxxxxxxxxxxxxx",
  API_URL:        "https://abcd1234efgh5678.lambda-url.ap-south-
1.on.aws",
};
```

Save the file. The orange DEMO MODE bar disappears next time you open the site.

Test it worked

Open `frontend/index.html` again. No orange bar. Create a real account with your own email, get the 6-digit code by email, confirm it, and sign in. You land on the student dashboard.

Make yourself a teacher

Every new account is a student. Turn yourself into a teacher in CloudShell:

```
aws cognito-idp admin-add-user-to-group \
  --user-pool-id ap-south-1_XXXXXXXXX \
  --username your.email@college.edu \
  --group-name Teacher
```

Sign out and sign back in — the new group only appears in a fresh login token. You now land on the teacher dashboard.

Common problems

Problem	Fix
Still shows the orange bar	`config.js` was not saved, or `API_URL` is still empty.
"Failed to fetch"	Wrong `API_URL`, or you left the slash on the end, or CORS is not set on the Function URL.
"Token is for another app"	`APP_CLIENT_ID` does not match the pool. Recheck Step 4's output.
Still a student after adding the group	You must sign out and sign in again.
Camera does not open	Expected. Opening a file directly is `file://`, not `https://`. This is fixed in Step 9.

STEP 8 — Put the website on S3

What: Upload the four frontend files to an S3 bucket.

Why: S3 is the cheapest place on AWS to keep files. Our whole site is about 100 KB.

Cost: Storing 100 KB is far less than one paisa a month. Even where the 5 GB S3 allowance does not apply, this rounds to ₹0.00 on your bill.

Do this

1. Console → search **S3** → **Create bucket**
2. Bucket name: something globally unique, e.g. `saurav-qr-attendance-2026`
3. Region: **Asia Pacific (Mumbai)**
4. **Leave "Block all public access" TICKED.** We are not making the bucket public — CloudFront will read it privately. This is both safer and free.
5. **Create bucket**
6. Open the bucket → **Upload**
7. **Add files**: `index.html`, `teacher.html`, `student.html`
8. **Add folder**: the `assets` folder
9. **Upload**

Test it worked

The bucket contains three `.html` files and an `assets` folder with `app.js`, `config.js` and `styles.css`. Four files inside `assets`, three at the top. Seven objects total.

Common problems

Problem	Fix
"Bucket name already exists"	Bucket names are global across all AWS customers. Add your roll number.
Uploaded but assets folder is flat	Use **Add folder**, not Add files, so the `assets/` path is kept.

STEP 9 — Put CloudFront in front (this is what gives you https)

What: A CloudFront distribution pointing at your S3 bucket.

Why: Two reasons, and the first one is non-negotiable:

1. **Phone browsers refuse to give a website camera access unless the page is on https.** No https, no scanning, no project. CloudFront gives you a free https address and certificate with no domain purchase.
2. It caches your files at edge locations so the site loads fast during your demo.

Cost: CloudFront's always-free tier covers 1 TB of data transfer out and 10 million requests per month. Your demo will use a few megabytes.

Do this

1. Console → search **CloudFront** → **Create distribution**
2. **Origin domain**: click the box and pick your S3 bucket from the list
3. **Origin access**: choose **Origin access control settings (recommended)**
   - Click **Create new OAC** → keep the defaults → **Create**
4. **Viewer protocol policy**: **Redirect HTTP to HTTPS**
5. **Allowed HTTP methods**: **GET, HEAD**
6. **Web Application Firewall**: choose **Do not enable security protections** *(WAF costs about $5 a month. Do not enable it.)*
7. **Price class**: choose **Use only North America and Europe**? **No** — choose **Use all edge locations**, or the cheapest class if offered. Either is inside the free tier at your volume.
8. **Default root object**: type `index.html`
9. **Create distribution**

Now S3 must be told to trust CloudFront:

10. A yellow banner appears saying the bucket policy needs updating → click **Copy policy**
11. Click **Go to S3 bucket permissions**
12. **Bucket policy** → **Edit** → paste → **Save changes**

Wait about 5 minutes for the status to change from **Deploying** to a date.

Get your website address

CloudFront gives you a **Distribution domain name** like:

`d1a2b3c4d5e6f7.cloudfront.net`

Your site is `https://d1a2b3c4d5e6f7.cloudfront.net`

Tighten CORS now that you have a real address

1. Go back to **Lambda** → your function → **Configuration** → **Environment variables**
2. Change `ALLOWED_ORIGIN` from * to `https://d1a2b3c4d5e6f7.cloudfront.net`
3. Save
4. **Configuration** → **Function URL** → **Edit** → change **Allow origin** from * to the same address → Save

This means only your own website can call your API.

Test it worked

Open `https://d1a2b3c4d5e6f7.cloudfront.net` on your **phone**. You should see the sign-in page with a padlock in the address bar. Sign in as a student, go to **Scan**, tap **Open camera** — the phone asks for camera permission. Say yes. The camera opens.

That moment is the whole project.

Common problems

Problem	Fix
`AccessDenied` XML page	The bucket policy was not pasted. Redo steps 10–12.
Blank white page	Default root object is missing. Edit the distribution → General → set `index.html`.
Old version keeps showing	CloudFront caches. Go to the **Invalidations** tab → **Create invalidation** → enter `/*`. The first 1,000 invalidation paths per month are free.
"Failed to fetch" after tightening CORS	The origin must match exactly — `https://`, no slash at the end, no typo.
Camera still blocked	Check the address bar starts with `https://`. In Chrome, tap the padlock → Permissions → Camera → Allow.

STEP 10 — Test everything, then rehearse the demo

Full test checklist

Create three accounts: one teacher (you), and two students (use a friend's phone, or a second email).

#	Test	Expected result
1	Register a new student, confirm by email code	Lands on the student dashboard
2	Teacher creates class "CSE 3A"	Class appears with a 6-letter join code
3	Student enters that join code	"Joined CSE 3A"
4	Teacher adds the second student by email	Name appears in the roster
5	Teacher starts a 15-minute session	QR appears, countdown ticks 10→0, QR changes
6	Student scans	"Attendance marked successfully", name appears on the teacher's screen within 4 seconds
7	**Same student scans again**	"You are already marked present" — and the record count does **not** go up
8	Screenshot the QR, wait 25 seconds, scan the screenshot	**"This QR code has expired"**
9	A student who never joined the class scans	"You are not enrolled in this class"
10	Teacher closes the session, student scans	"This attendance session is closed"
11	Student opens **My attendance**	Percentage and history shown
12	Teacher opens **Report** → Download CSV	CSV file downloads with every student's percentage
13	Open your Lambda → Monitor → CloudWatch logs	You can see each request; no red errors
14	Billing → Bills	**$0.00**

If test 7 and test 8 both pass, your security story is real, not decorative.

The demo script for your examiner

Rehearse this. It should take four minutes.

1. "This is the teacher dashboard." Log in, show CSE 3A.
2. "I start a session." Click Start. **The QR fills the screen with a countdown.**
3. Two phones scan. Names appear live.
4. "Now watch what happens if a student scans twice." Scan again → *Already marked present.* "The pair of session and student is the database key, so a second row physically cannot exist."
5. **The winning moment.** "Students sometimes screenshot the code and send it to absent friends." Take a screenshot. Wait 20 seconds. Scan the screenshot. → **"This QR code has expired."** "The code is signed with a secret only my Lambda knows, and it is rebuilt every 10 seconds."
6. Show the student dashboard: percentage, history.
7. Show the CSV export.
8. Open the AWS Billing console. "Total cost: zero rupees."

STEP 11 — Watch the cost

Do this once a week while the project is live.

1. **Billing and Cost Management → Bills** — the number should be $0.00
2. **Billing → Free tier** — shows how much of each allowance you have used. Note it lags by about 24 hours, so it is a rear-view mirror, not a live gauge. Your $1 budget from Step 1 is the real alarm.
3. **DynamoDB → any table → Monitor** — consumed capacity should sit far below your provisioned 20.

Things that would cost money — never click these

Trap	Roughly what it costs
NAT Gateway	₹2,800/month. The most common surprise bill on AWS. Our design uses no VPC, so you should never see this.
EC2 or RDS	Hourly, and for new accounts it comes out of your credits
Elastic IP	Charged even when unused
DynamoDB on-demand mode or Standard-IA class	Billed from the first request
DynamoDB Accelerator (DAX)	Not in the free tier at all
Route 53 hosted zone	About ₹42/month. This is the "no paid domain" rule in practice.
SNS / SMS messages	Real money per message
AWS WAF	About $5/month per web ACL
CloudWatch detailed metrics	$0.30 per metric per month, and they

Trap	Roughly what it costs
	multiply fast
Anything in AWS Marketplace	Usually a paid subscription
Support plan above Basic	Monthly fee

One small housekeeping job

Console → **CloudWatch** → **Log groups** → `/aws/lambda/qr-attendance-api` → **Actions** → **Edit retention setting** → **7 days**.

By default logs are kept forever. Yours will be tiny either way, but this is good practice and worth mentioning in your report.

STEP 12 — Delete everything when you are done

Why: The only guaranteed way to never be billed is to own nothing.

When: After your marks are in. Not before — examiners sometimes ask to see it again.

1. Open **CloudShell**
2. Open `aws/99-delete-everything.sh`, edit the three lines at the top with your bucket name, function name and user pool ID
3. Paste the whole file into CloudShell, type `DELETE` when asked

Then two things the script cannot do:

4. **CloudFront** → select your distribution → **Disable** → wait about 15 minutes until it says Disabled → **Delete**
5. **IAM** → **Roles** → find `qr-attendance-api-role-...` → **Delete**

Keep the $1 budget. It costs nothing and it protects you forever.

Finally: **Billing** → **Bills** → confirm **$0.00**.

Quick reference: what every file does

```
qr-attendance/
├── backend/
│   └── index.mjs ................. the entire backend. Paste into
Lambda.
├── frontend/
│   ├── index.html ................ sign in / register
│   ├── teacher.html .............. classes, sessions, rotating QR,
reports
```

```
│   ├── student.html ............... join class, scan, attendance
history
│   └── assets/
│       ├── config.js ............... THE ONLY FILE YOU EDIT
│       ├── app.js .................. login, API calls, and the demo
backend
│       └── styles.css .............. the look
├── aws/
│   ├── 01-create-tables.sh ........ makes the 5 DynamoDB tables
│   ├── 02-create-cognito.sh ....... makes the login system
│   ├── lambda-dynamodb-policy.json  the permission Lambda needs
│   └── 99-delete-everything.sh .... cleanup
└── docs/
    ├── SETUP-GUIDE.md ............. this file
    ├── ARCHITECTURE.md ............ diagrams and the request flow
    └── VIVA-QUESTIONS.md .......... likely examiner questions with
answers
```

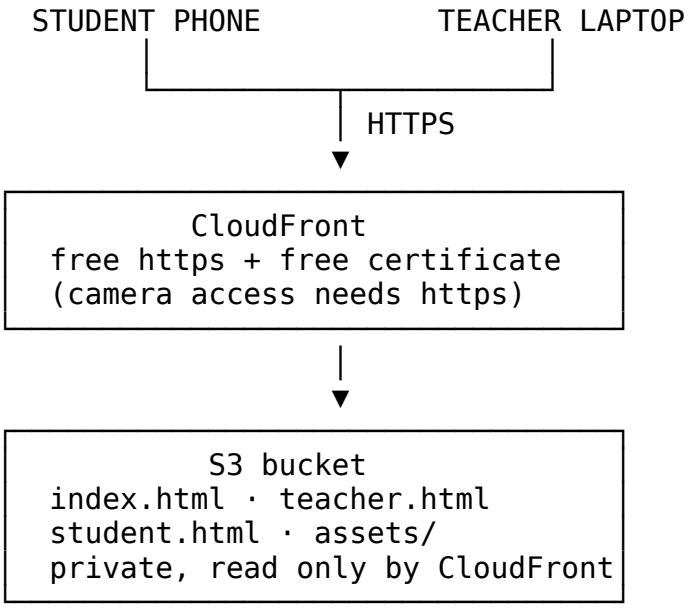
Made by Saurav

Architecture & Design

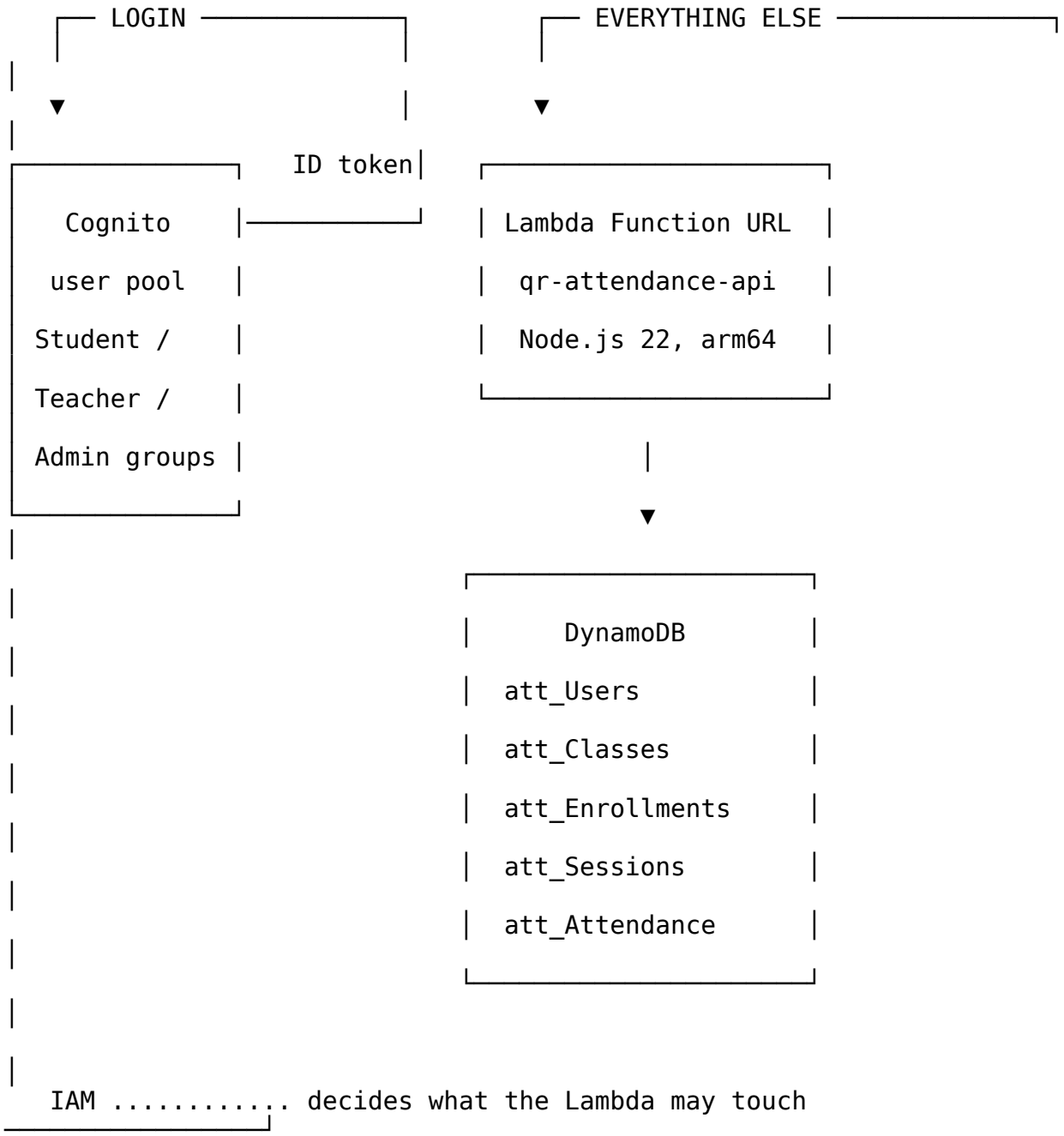
QR Code Based Attendance System — Made by Saurav

---

1. System architecture



The browser also makes two kinds of call on its own:



---

2. Why each service

Service	Why it is here	What we rejected instead
S3	Cheapest storage for a static site	EC2 — a server running 24×7 that we would pay for

Service	Why it is here	What we rejected instead
CloudFront	Free HTTPS. Phone cameras will not work without it	S3 website hosting — HTTP only
Cognito	Passwords, email confirmation, reset, secure tokens	Hand-rolled auth — the usual source of security holes
Lambda	Runs only on request, always-free allowance	A running Node server
Function URL	HTTPS endpoint with no extra charge	API Gateway — its free tier is a 12-month offer
DynamoDB	No server, always-free allowance, key design blocks duplicates	RDS/MySQL — a database instance billing by the hour
IAM	Lambda can touch five tables and nothing else	A wide-open role
CloudWatch	Where errors go so you can debug	Guessing

3. Database design

att_Users

Field	Type	Notes
userId	String	**Partition key.** The `sub` from the Cognito token
email	String	GSI `email-index` so a teacher can add a student by email
name	String	
role	String	student / teacher / admin
lastSeen	Number	

att_Classes

Field	Type	Notes
classId	String	**Partition key**
name, subject	String	
teacherId, teacherName	String	
joinCode	String	6 characters, no confusing 0/O or 1/I
createdAt	Number	

att_Enrollments

Field	Type	Notes
classId	String	**Partition key**
studentId	String	**Sort key**
studentName, studentEmail	String	

GSI `student-index` on studentId, so a student can list their own classes.

att_Sessions

Field	Type	Notes
sessionId	String	**Partition key**
classId, className, teacherId	String	
status	String	OPEN or CLOSED
startedAt, expiresAt	Number	milliseconds

att_Attendance

Field	Type	Notes
sessionId	String	**Partition key**
studentId	String	**Sort key**
studentName, studentEmail, classId, className	String	
markedAt	Number	
method	String	QR or MANUAL

GSI `student-index` on studentId + markedAt, for a student's history.

The duplicate-prevention design. Because the primary key is the *pair* `sessionId + studentId`, that pair can exist at most once. The write also carries `ConditionExpression: attribute_not_exists(sessionId)`, so DynamoDB itself rejects the second attempt. Two phones tapping at the identical millisecond still cannot create two rows.

Free-tier capacity budget

Table / index	Read	Write
att_Users	2	2
att_Users / email-index	2	2
att_Classes	2	2
att_Enrollments	3	3
att_Enrollments / student-index	2	2

Table / index	Read	Write
att_Sessions	3	3
att_Attendance	3	3
att_Attendance / student-index	3	3
Total	**20**	**20**
Free allowance	25	25

All tables are **provisioned capacity, Standard table class** — the only combination the free allowance covers.

---

4. How the rotating QR code works

The QR picture is never made on AWS and never stored anywhere. It is drawn in the browser from a short piece of text.

```
window   = floor(current time in ms / 10000)      → changes every 10 seconds
body     = sessionId + "." + window
signature= HMAC-SHA256(body, QR_SECRET) shortened
token    = body + "." + signature
```

`QR_SECRET` lives only in a Lambda environment variable.

When a scan arrives, Lambda:

1. Recomputes the signature. If it does not match, the QR was not made by us → rejected.
2. Compares the window to the current one. More than one window apart → expired.

One window either side is allowed, so real validity is about 20 seconds. That is enough for a slow camera and a phone with a slightly wrong clock, and far too short to forward a screenshot to a friend across campus.

This is the same idea as a Google Authenticator number.

---

5. Security model

Requirement	How it is met
Authentication	Cognito issues a signed ID token. Lambda downloads Cognito's public keys and verifies the RSA signature, expiry, audience and issuer on every single request.
Authorization	The role comes from the `cognito:groups` claim **inside the signed token** — the browser cannot fake it. Teacher-only routes

Requirement	How it is met
	call `requireTeacher()`.
Unique session	Every session gets a random ID. A student can only be marked present against a session that exists and is OPEN.
Session expiry	Two layers: the 10-second QR window, and the session's own `expiresAt`.
Duplicate prevention	Composite primary key plus a conditional write. Enforced by the database, not by application logic.
Server-side validation	All six checks run inside Lambda. Nothing sent by the phone is trusted.
Secure database access	An IAM inline policy grants five specific table ARNs and five actions. Nothing else in the account is reachable.
QR sharing / reuse	The signed, rotating token makes a forwarded screenshot useless within seconds. Enrolment is also checked, so a student from another class cannot mark themselves present.

Honest limitations

Worth saying yourself before the examiner says it:

- A student **inside the room** can still relay a live code to a friend over a video call within the 20-second window. Fixing that properly needs a location check or Bluetooth proximity. The code has a place to add a GPS check.
- Function URL auth type is NONE, so anyone can reach the endpoint — but without a valid Cognito token every request is rejected with 401. Authentication is done by our code, deliberately, so we do not pay for API Gateway.
- Two students could share one login. That is a password problem, not a QR problem, and no attendance system solves it without biometrics.

6. Cost model

Service	Free allowance	What this project uses
Lambda	1M requests + 400,000 GB-s per month, always free	~50,000 requests total
DynamoDB	25 GB, 25 RCU, 25 WCU per month, always free	~1 MB, 20 RCU/WCU
Cognito	10,000 monthly active users, does not expire	~5
CloudFront	1 TB out + 10M requests per	under 1 GB

Service	Free allowance	What this project uses
	month, always free	
S3	5 GB (disputed for new accounts)	~100 KB — rounds to ₹0.00 either way
CloudWatch Logs	5 GB ingest per month	a few MB
Function URL	no additional charge	—
IAM, Budgets (first 2)	free	—

Account-level warning. A new AWS account on the Free plan ends at six months or when the $100 + $100 credits run out, whichever comes first — and then closes automatically, with 90 days to recover data. Note your signup date.

Made by Saurav

Viva Questions & Answers

QR Code Based Attendance System — Made by Saurav

Read this the night before. The answers are written the way you would actually say them out loud.

---

1. Why did you use serverless instead of a normal server?

A normal server runs 24 hours a day whether anyone uses it or not, so you pay for idle time and you have to patch and secure it yourself. Lambda runs only when a request arrives. For an attendance system that is busy for five minutes a day, that is the right shape — and it fits inside AWS's always-free allowance of one million requests a month.

---

2. How does your system stop proxy attendance?

Four ways, layered.

First, the QR code on the teacher's screen changes every ten seconds, so a screenshot forwarded to a friend has expired before it arrives. Second, the code is signed with a secret key that only my Lambda function knows, so nobody can make a fake one. Third, the student has to be logged in with their own Cognito account — the code alone marks nobody present. Fourth, the system checks the student is actually enrolled in that class.

I should be honest about the limit: a student in the room could still relay a live code over a video call within that twenty-second window. Fixing that fully needs a GPS or Bluetooth proximity check, which is the next thing I would add.

3. How exactly does the rotating QR code work?

It is the same idea as Google Authenticator. I take the session ID and the current ten-second time window, join them, and sign that with HMAC-SHA256 using a secret stored in a Lambda environment variable. The result is a short piece of text. The browser turns that text into a QR picture.

When a scan comes in, Lambda recomputes the signature. If it does not match, the code is fake. Then it checks the time window is still current. I allow one window either side for slow cameras and clock differences, so real validity is about twenty seconds.

The QR image is never generated on AWS and never stored anywhere, which is why the system costs nothing in storage.

---

4. How do you prevent duplicate attendance?

I did not solve it in application code, I solved it in the database design. The Attendance table's primary key is the *pair* of session ID and student ID. That pair can physically exist only once. On top of that the write carries a condition — `attribute_not_exists(sessionId)` — so DynamoDB rejects the second write itself.

That means even if two phones send the request in the same millisecond, only one row can be created. If it had been an `if (alreadyExists)` check in my code, there is a race condition where both requests read "not present" before either writes.

---

5. What is Cognito actually doing?

It stores users, enforces password rules, sends the email confirmation code, and issues a signed JSON Web Token when someone logs in. My website keeps that token and attaches it to every request.

The important part is that I do not trust the token blindly. Lambda downloads Cognito's public keys and verifies the RSA signature, the expiry time, the audience and the issuer on every single request. If any of those fail, the request is rejected with a 401.

---

6. Where is the user's role stored, and could a student fake being a teacher?

The role is a Cognito group — Teacher or Admin — and the group name travels inside the signed token as the `cognito:groups` claim. Because the token is signed by Cognito and verified by my Lambda, changing it would break the signature.

A student could edit their browser's JavaScript to *show* the teacher screen, but every teacher action calls a server route that checks the role again. They would just get "Only a teacher can do this."

---

7. Why did you not use API Gateway?

Cost, and I checked rather than assumed. API Gateway's free allowance of one million calls is a twelve-month offer, and current pricing guidance says it applies only to accounts created before July 2025 — newer accounts draw down credits and then pay a dollar per million HTTP API requests.

A Lambda Function URL gives the same HTTPS endpoint at no additional charge — you pay only the normal Lambda cost, which my project stays inside. I would use API Gateway if I needed rate limiting, API keys, or a custom authorizer, but for one small API it is a service and a charge I do not need.

---

8. Function URL auth type is NONE. Is that not insecure?

It means AWS does not check the caller for me. My own code does, on every request, before touching any data. So the endpoint is reachable by anyone, but useless without a valid Cognito token.

That is a deliberate trade: I moved the authentication check into my own code so I do not have to pay for API Gateway. It is the same check either way.

---

9. Why DynamoDB and not MySQL?

Three reasons. There is no server to run or pay for. The free allowance of twenty-five gigabytes is permanent rather than a trial. And the key design does the duplicate prevention for me.

The trade-off is that DynamoDB cannot do joins, so I do some work in code that SQL would do in a query — for example calculating the attendance percentage. At the size of one college that is fine.

---

10. What is a GSI and why do you need two?

A Global Secondary Index is a second way to look up the same data. My Enrollments table is keyed by class, so "who is in this class" is fast — but "which classes is this student in" would need a full scan. The GSI on studentId makes that fast too.

The thing to watch is cost: every GSI needs its own provisioned capacity, and it comes out of the same twenty-five free units as the tables. I budgeted twenty read and twenty write in total across five tables and two indexes, leaving five spare.

11. Why do you need CloudFront? Could you not just use S3?

S3 static website hosting only serves plain HTTP. Phone browsers refuse to give a website access to the camera unless the page is on HTTPS, so without CloudFront the scanner simply would not open. CloudFront gives me a free HTTPS address and certificate without buying a domain.

It also caches the files at edge locations so the page loads quickly, and its always-free tier covers a terabyte of transfer a month.

---

12. What did this cost you?

Zero. Everything is inside AWS's always-free allowances — Lambda, DynamoDB, Cognito and CloudFront all have permanent monthly limits that this project uses a tiny fraction of.

I also set a one-dollar budget alert before creating a single resource, and I deliberately avoided the services that generate surprise bills — NAT Gateway, RDS, Elastic IPs, Route 53 and SMS.

One thing I would flag: a new AWS Free plan account closes automatically after six months or when the signup credits run out. So this deployment has a shelf life unless I upgrade the account.

---

13. What would you do differently or add next?

Three things, in order.

A location check — have the browser send GPS coordinates and reject scans more than a hundred metres from campus. I left a hook for it. It would close the video-call relay gap.

Second, an offline queue, so if the college wifi drops mid-scan the phone retries when it reconnects.

Third, at real scale I would move to a single-table DynamoDB design and add API Gateway with rate limiting, because right now nothing stops someone hammering my Lambda.

---

14. Walk me through what happens when a student scans.

The camera reads the QR and gets a short text code. The browser sends that code to my Lambda Function URL with the student's login token in the Authorization header.

Lambda then runs six checks. Is the login token really signed by my Cognito pool. Was this QR code really made by me — I recompute the HMAC signature. Is the ten-second window still current. Is the session still open and not past its end time. Is this student enrolled in that class. And finally, does an attendance row already exist for them.

If all six pass, it writes one row to the Attendance table and returns success. The phone shows "Attendance marked successfully" and the teacher's screen picks up the new name within four seconds.

Every one of those checks is on the server. Nothing the phone sends is trusted.

---

15. Show me it failing.

(Have this ready — it is the strongest thirty seconds of the demo.)

Screenshot the QR. Wait twenty-five seconds. Scan the screenshot. The screen says **"This QR code has expired."**

Then scan the live code twice. The second time says **"You are already marked present"** and the attendance count does not move.

Made by Saurav